

Async Main in C# with Examples

In this article, I will discuss the **Async Main in C#** with Examples. Please read our previous article discussing the **Thrown Expression in C#** with Examples. From C# 7.1, defining the Main Method as Async is possible. At the end of this article, you will understand **What Exactly Async Main is in C# 7** and **When and How to use Async Main** with Examples.

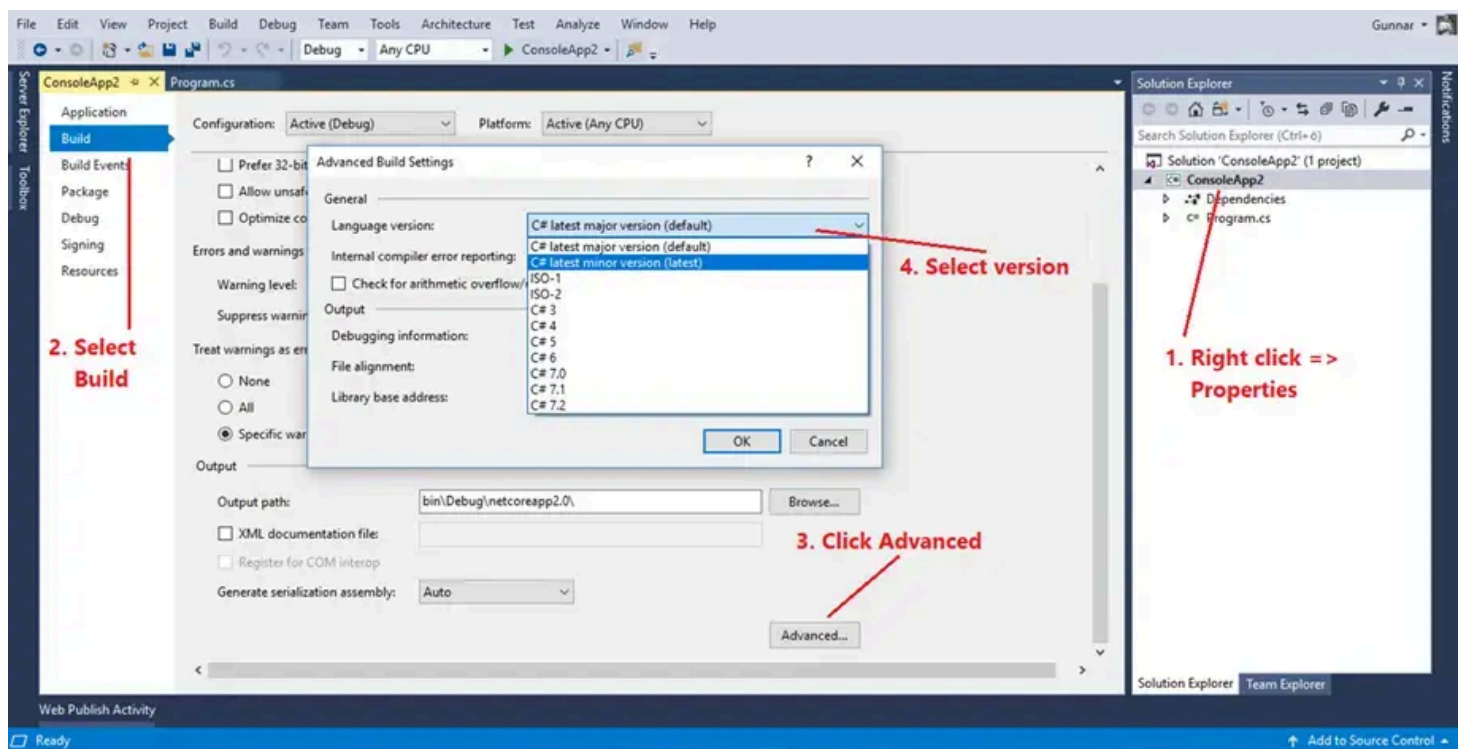
Async Main is in C# 7:

In C#, asynchronous programming performs long-running operations without blocking the main thread of execution. The `async` keyword in C# defines an asynchronous method, which allows you to use the `await` keyword to call other asynchronous methods without blocking. This is particularly useful in desktop, web, and mobile applications where a responsive user interface is important.

Starting with C# 7.1, you can create an `async Main` method as well, which allows you to use `await` at the entry point of your application. This is helpful when performing asynchronous operations as soon as the application starts without blocking the main thread.

Activating C# 7.1 features

Follow the steps below to activate C# 7.1 Features in Visual Studio 2017 or lower version. For Higher Versions of Visual Studio, it is available automatically.



For those who need some more help, the steps are described here:

1. Right-click on the project name and select Properties from the context menu
2. Select the Build tab from the left
3. Scroll down and click the Advanced button to open advanced build settings

4. From the Language version drop-down, select the C# version you are interested in

C# Latest Minor Version (latest) is the option to select if you want the latest version of C#.

Main Method in C#

Like other programming languages, the C# program starts from the Main method with the following properties.

1. The C# entry point method must be static,
2. The name of the method must be the Main
3. The return type of this method can be either void or int.
4. It can have one parameter of a string array containing any command-line arguments.

Before C# 7.1, four overloaded versions are considered valid signatures for the Main method in C#, as shown below.

```
public static void Main();  
public static int Main();  
public static void Main(string[] args);  
public static int Main(string[] args);
```

Before C# 7.1, when we wanted to call the async method from within the Main() method, we needed to add some code, but now, the C# compiler does it for us. Let's try to understand how to call the async method from Main in C# before C# 7.1 with an example. Please have a look at the below code for a better understanding.

```
using System;  
using System.Threading.Tasks;  
namespace AsyncMainDemo  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            Console.WriteLine("Before C# 7.1, To use async method");  
            Console.WriteLine($"Main Method execution started at {System.DateTime.Now}");  
            //Calling Async Method  
            //We cannot use await as the Main method is not async  
            //Hence using Wait Method to wait for the completion of the SomeAsyncMethod()  
            SomeAsyncMethod().Wait();  
            Console.WriteLine($"Main Method execution ended at {System.DateTime.Now}");  
            Console.WriteLine("Press any key to exist.");  
            Console.ReadKey();  
        }  
        //Async Method  
        static async Task SomeAsyncMethod()  
        {  
            await Task.Delay(2000);  
        }  
    }  
}
```

```

    }
}
}

```

Output:

```

Before C# 7.1, To use async method
Main Method execution started at 13-06-2018 12:56:33
Main Method execution ended at 13-06-2018 12:56:35
Press any key to exist.

```

Async Main in C# 7:

From C# 7.1, the Main() method, which is the application's entry point, can be declared async. Before C# 7.1, the Main() method can have a return type as either void or int; however, now, it also supports **Task** and **Task<int>**. So, From C# 7.1, we now have eight overload versions that are considered valid signatures for the **Main()** method, as shown below.

```

public static void Main();
public static int Main();
public static void Main(string[] args);
public static int Main(string[] args);
public static Task Main();
public static Task<int> Main();
public static Task Main(string[] args);
public static Task<int> Main(string[] args);

```

Example to Understand Async Main Method in C# 7

In C#, starting with C# 7.1, you can use the async modifier with the Main method to create an asynchronous entry point for your console application. This allows you to write asynchronous code directly in the Main method.

Please look at the example below for a better understanding of the Async Main Method. In the below example, we have declared the Main method as async and Task as the return type. Then we call the SomeAsyncMethod using the await operator as now the Main method is an async method.

```

using System;
using System.Threading.Tasks;
namespace AsyncMainDemo
{
    class Program
    {
        static async Task Main(string[] args)
        {
            Console.WriteLine("From C# 7.1, Async Main Method");
            Console.WriteLine($"Main Method execution started at {System.DateTime.Now}");
            //Calling Async Method using await as now the Main method is async
            await SomeAsyncMethod();
        }
    }
}

```

```

        Console.WriteLine($"Main Method execution ended at {System.DateTime.Now}");
        Console.ReadKey();
    }
    //Async Method
    static async Task SomeAsyncMethod()
    {
        await Task.Delay(2000);
    }
}

```

Output:

```

From C# 7.1, Async Main Method
Main Method execution started at 06-07-2023 21:40:44
Main Method execution ended at 06-07-2023 21:40:46

```

As you can see in the above example, the Task.Delay is adding a 2-second delay in the program execution. Now, C# 7.1 syntax is more straightforward and easy to use. In the above example, you can see how we use the Task within the Main method.

Example to Understand Async Main Method in C# 7 with Task<int>

Let's take another example where we will use Task<int>. Here, we will call another async method, AdditionAsync, from the Main method.

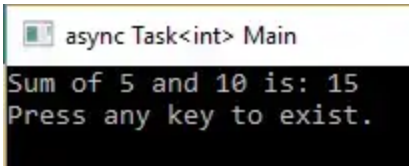
```

using System;
using System.Threading.Tasks;
namespace AsyncMainDemo
{
    class Program
    {
        public static async Task<int> Main(string[] args)
        {
            Console.Title = "async Task<int> Main";
            int number1 = 5, number2 = 10;
            Console.WriteLine($"Sum of {number1} and {number2} is: {await
AdditionAsync(number1, number2)}");
            Console.WriteLine("Press any key to exist.");
            Console.ReadKey();
            return 0;
        }
        private static async Task<int> AdditionAsync(int no1, int no2)
        {
            await Task.Delay(2000);
            return no1 + no2;
        }
    }
}

```

```
}  
}
```

Output:



```
async Task<int> Main  
Sum of 5 and 10 is: 15  
Press any key to exist.
```

When to use the Async Main Method in C#?

The async Main method in C# is used as the entry point for an asynchronous application. This feature was introduced in C# 7.1 and allows the Main method to have the async modifier, enabling await within it. This is particularly useful in a variety of scenarios:

I/O-Bound Operations

When your application performs I/O-bound operations at startup, such as:

- Reading from or writing to files.
- Making network requests (e.g., calling web APIs).
- Database operations.

Using async Main allows these operations to be performed asynchronously, improving your application's responsiveness and scalability.

Asynchronous Libraries

If your application relies on libraries that expose asynchronous APIs, you can directly call these APIs from the Main method using await. This is cleaner and more straightforward than using .GetAwaiter().GetResult() or .Result, which can lead to deadlocks in certain contexts.

Console Applications

In console applications that perform asynchronous operations, using an async Main method simplifies the codebase. It's especially beneficial in scenarios like:

- Command line tools that interact with HTTP services.
- Applications that require parallel processing or task coordination at startup.

Simplifying Code

Using async Main simplifies the code structure by allowing the use of await at the top level of the application. It avoids the need for additional async methods solely to use await.

Things to Consider

- **Application Type:** Primarily useful in console applications. In other applications like web apps, the framework typically handles asynchrony for you.
- **Error Handling:** Ensure proper error handling in the Main method. Unhandled exceptions in asynchronous methods can terminate your application.
- **Compatibility:** async Main is available from C# 7.1 onwards, so ensure your project is configured to use an appropriate language version.

In the next article, I will discuss [Exception Handling in C#](#) with Examples. In this article, I try to explain the **Async Main Method in C# 7** with Examples. I hope this article will help you with your needs. I would like to have your feedback. Please post feedback, questions, or comments about this Async Main Method in the C# 7 article.